Member 4 — Dashboard and Storage

Member placeholder: [MEMBER 4 NAME]

Proposed viva responsibility: browser dashboard behavior, exact clinical labels, null/gap-aware rendering, history charts and CSV, SQLite observation storage, schema migration, retention and device isolation.

Authorship disclaimer: this document maps the repository's dashboard/storage capabilities to a proposed HOD responsibility. It does not prove that [MEMBER 4 NAME] authored these files. The team must confirm authorship; use **"the team implemented"** for source-backed shared work.

A. Responsibility scope

A.1 One-sentence scope

For this discussion, [MEMBER 4 NAME] is responsible for explaining how valid telemetry becomes an honest dashboard and a gap-preserving local history: exact labels, separate actual-sensor and clinical-wrist values, null instead of zero, no line across missing data, and blank CSV cells for gaps.

A.2 Included in this responsibility

- index.html page structure and the visible clinical labels.
- The four active metric cards and the separate core/fever/actual-TMP cards.
- assets/dashboard_core.js pure helpers and their tests.
- assets/dashboard.js live SSE consumption, history fetching, chart drawing, tooltips, baseline/device views, CSV download.
- The dashboard's independent status/temperature checks.
- telemetry_store.py SQLite schema, additive migration, measurement acceptance, retention, bucket history and gaps.
- Storage of clinical wrist values only with verified contact and matching actual evidence.
- CSV contents and missing-value policy.

- Local browser/storage privacy and the limits of the "local only" model.
- Evidence status: source, Node tests, Python history tests, build; no full browser E2E.

A.3 Outside primary scope — but required knowledge

- Firmware JSON producer: Member 1.
- TMP117/MAX/MPU hardware validity: Member 2.
- Bridge validation/API/SSE: Member 3.
- Temperature policy and baseline/alert algorithms: Member 5.

Member 4 must know why the clinical wrist value is null while actual TMP117 is displayed. A correct label with a wrong value is still a failure.

A.4 Key files

File	Role
index.html	Three dashboard views and metric/temperature card markup
assets/dashboard.js	DOM/SSE/history/chart/CSV/baseline/device behavior
assets/dashboard_core.js	Pure helpers for status, values, temperature assessment, gaps and CSV
assets/dashboard.css	Layout/theme/responsive presentation
telemetry_store.py	SQLite observation/history storage and acceptance rules
personal_baseline.py	Baseline/enrollment/events consumed by dashboard APIs
tests/dashboard_core.test.js	10 Node tests for pure dashboard helpers
tests/test_history.py	Store acceptance/migration/gap/device tests
telemetry_bridge.py	APIs and SSE producer
README.md	Label, storage, history and CSV policy

B. Achievements attributable to this area

Achievement	What the repository contains	Evidence

Achievement	**What the repository contains**	**Evidence**
Exact clinical temperature labels	Runtime labels **Wrist Skin Temperature (°C)** and **TMP117 Sensor Temperature (°C)**	[SOURCE] dashboard.js metric metadata; dashboard_core.js specs; index.html
Separate actual/clinical display	Actual TMP117 card/evidence separate from clinical wrist card	[SOURCE] renderEnvironment(); [TEST] Node cases
Null/status-aware cards	Missing/invalid/offline/no-finger values show notes, not zero	[SOURCE] metricStatus(), metricValue(), renderReadings(); [TEST]
Gap-aware chart	New segment after a null bucket	[SOURCE] gapAwarePath(); [TEST]
CSV with blank gaps	Active metrics exported; missing cells blank	[SOURCE] csv(); [TEST]
Four active metrics	HR, SpO2, wrist skin temperature, motion	[SOURCE] dashboard.js; [TEST] Node
Clear core/fever unavailability	Core/fever cards plus exact message	[SOURCE] temperatureAssessment(), renderEnvironment()
No SHT40/MLX UI	Installed inventory only; optional modules absent	[SOURCE] sensorHealth()/hardware view; [TEST]
SQLite history	Valid observations, status context, raw wrist code, 30-day retention	[SOURCE] TelemetryStore; [TEST] 62 Python suite
Additive migration	Old DBs gain columns without rewriting	[SOURCE] MIGRATION_COLUMNS, _migrate(); [TEST]
Device isolation	History queries filter by device_id	[SOURCE]; [TEST] persistence/isolation
Browser-local review	Review note stored locally, not sent as clinical data	[SOURCE] renderReview()/local storage behavior
Reproducible host tests	10 Node dashboard tests; Python history tests	[TEST] fresh 2026-10-29

B.1 What the area does not establish

- That the dashboard renders correctly in every real browser/device.
- That the full DOM, SSE reconnect, chart and CSV download were tested end-to-end in a browser.
- That CSS is fully accessible/WCAG-compliant; no formal audit.
- That SQLite survives every disk-full/corruption/concurrency scenario.
- That the local host is secure for real patient data.
- That the chart's browser clock and stored server timestamps always agree.
- That history contains actual TMP117 sensor temperature; it deliberately does not store it as the clinical wrist metric.

C. Detailed code walkthrough

C.1 Page structure and exact labels

`index.html` has three views:

1. **Overview:** live metric cards, emergency banner, core/fever cards, actual TMP117 card, environment/temperature evidence, trend chart, baseline preview, observations.
2. **Baseline:** enrollment, coverage calendar, reference table, observations/review.
3. **Device & sensors:** connection/board facts and sensor inventory, plus the temperature note.

The four active metric tabs are:

- Heart rate;
- Blood oxygen;
- **Wrist Skin Temperature (°C)**;
- Movement.

The temperature area also contains separate cards for:

- **Core temperature** — value "Unavailable"; note is the exact runtime fever message.
- **Fever screening** — "Not available"; no validated core rule.

● **TMP117 Sensor Temperature (°C)** — actual sensor reading, explicitly "not core temperature."

The runtime labels come from assets/dashboard.js metadata and DashboardCore.SPECS; the static HTML provides the initial accessible labels. The exact required cardinal labels are implemented, even though the OLED uses compact text.

Important: the actual TMP117 card is not the clinical wrist card. The clinical card shows no number while contact is unverified. Do not point at the actual value and call it wrist temperature.

C.2 Dashboard core — pure helpers

assets/dashboard_core.js is separated from DOM code so it can be tested by Node.

Key functions:

Function	Behavior
finite(value)	Returns a finite number or null; rejects strings/booleans/NaN/Infinity
rawCode(value)	Integer 0..0xFFFF or null
rawCelsius(raw)	Signed 16-bit code × 0.0078125
timestamp(value)	Non-negative integer or null
sensorOnline(packet, sensor)	packet.sensors[sensor] === true
metricStatus(packet, key)	Presence/status/raw/range/matching-evidence validation
metricValue(packet, key)	Returns a finite value only if status is valid and sensor/finger conditions hold; otherwise null
temperatureAssessment(packet)	Builds actual/wrist/core/fever assessment; core/fever always unavailable
activeSensorCount(packet)	Counts only max, mpu, tmp117
sensorHealth(packet, key)	Returns status/error from sensor_health or a not-available/warming fallback
statusLabel(status)	Human-readable labels; unknown → "Not available"
gapAwarePath(points, key, x, y)	SVG path with M after a null gap
csv(points)	CSV header/rows for active metrics, blank for missing

Function	Behavior

metricStatus() for actual TMP117:

- requires sensor present and a recognized status;
- rejects unverified for the actual sensor (it is not a contact status);
- for valid, requires a finite value, raw code, timestamp, non-stuck raw, value within -55..150, and raw conversion within 0.02 °C;
- otherwise returns sensor_error rather than displaying a bad value.

metricStatus() for clinical wrist:

- requires wrist_contact_status === "verified";
- for valid, requires actual TMP117 status valid, wrist value/raw/timestamp, matching actual raw and timestamp, raw conversion within 0.02 °C, actual value within 0.02 °C, value in -10..60 °C, non-stuck raw;
- with unverified contact returns unverified; non-null leaked evidence becomes sensor_error.

metricValue() also returns null for optical metrics when finger_on is not true. This is why a stale old HR is not shown as live.

temperatureAssessment() returns:

tmp117_sensor_temperature_c/status/raw/timestamp

wrist_contact_status

wrist_skin_temperature_c/status/raw/timestamp

estimated_core_temperature_c: null

estimated_core_temperature_status: not_available

estimated_core_available: false

fever_screening_status: not_available

fever_screening_available: false

fever_screening_input: { measurement: wrist_skin_temperature, ... }

fever_screening_rule: null

uncertainty

limitations

message: "Core temperature unavailable — confirmation required"

measured

It never computes a core value or a fever rule.

C.3 Live consumption — consume() and startStream()

startStream() opens new EventSource('/api/stream'). On each message:

consume(JSON.parse(e.data));

consume(payload):

- reads payload.telemetry, payload.status, payload.comparisons;
- only updates the displayed packet if connected, the telemetry object has device_id/vitals/sensors, and _received_at differs from the previous packet;
- this prevents repeated rendering of the same snapshot;
- updates connection state and re-renders readings;
- if disconnected, clears state.packet and comparisons and renders unavailable cards.

On EventSource error, it clears the packet, sets a connection error and renders unavailable data. It does not retain a stale number.

renderReadings() updates the four metric cards, then calls renderEnvironment() and renderHardware(). Notes are status-driven:

- no packet → "No live data";
- sensor missing → "Not connected";
- error/stale → human status;
- optical with no finger → "No finger detected";
- non-valid status → status label;
- valid but null → "Signal settling";

● valid value → baseline comparison or "Measured now."

renderEnvironment() fills core/fever/actual cards and a long evidence line containing actual sensor value/status/raw/timestamp, contact, clinical wrist input, quality, wrist raw/timestamp, and rule. It uses the exact runtime message for core/fever.

C.4 History and chart

fetchHistory():

● increments state.historyRequest and stores the request number;

● builds /api/history?seconds=<range>&device_id=<id>;

● fetches JSON;

● ignores an out-of-order response if request !== state.historyRequest;

● sets state.history = data.points || [];

● labels the resolution from data.bucket_seconds;

● calls drawChart().

drawChart():

● uses the selected metric, valid points and optional baseline model;

● computes the SVG viewBox from the panel size;

● draws axes, grid, baseline band and path;

● uses DashboardCore.gapAwarePath() so a null bucket starts a new M segment;

● sets an accessible aria-label with metric and recorded interval count;

● updates tooltip/cursor via inspectChart().

gapAwarePath() is only a few lines but is the key visual honesty:

```
for (const point of points || []) {
  const value = finite(point?.[key]);     // null for missing/invalid
  if (value === null) { started = false; continue; }
  path += `${started ? 'L' : 'M'}${x(point.time)},${y(value)} `;
```

started = true;

}

A missing bucket produces a new M, not a line drawn through the gap.

fetchHistory() runs initially and every 5 s; fetchBaseline() runs initially and every 15 s. Live SSE handles faster updates.

C.5 CSV export

exportButton filters state.history to rows with samples, calls core.csv(rows), creates a Blob and downloads pulsecare-YYYY-MM-DD.csv, then revokes the URL.

DashboardCore.csv() columns:

timestamp, heart_rate_bpm, spo2_percent,

wrist_skin_temperature_c, motion_g, samples

- Missing values are empty strings.
- The export contains **clinical wrist** data, not the actual TMP117 sensor temperature.
- It contains only active metrics; legacy temp_object is not exported.
- Rows with zero samples are filtered by the caller.
- The timestamp is ISO from the bucket time.

C.6 Storage — telemetry_store.py

TelemetryStore opens SQLite with:

- row_factory = sqlite3.Row;
- PRAGMA journal_mode=WAL;
- check_same_thread=False;
- an RLock for operations;
- tables observations, enrollments, events.

The observations table stores:

- device_id, server recorded_at, uptime_s;

- heart_rate, spo2, wrist_skin_temperature, wrist_skin_temperature_raw;
- legacy temp_object;
- motion_g, raw_ir, raw_red, finger_on;
- sos, fall;
- statuses for HR/SpO2/wrist/motion.

_migrate() uses MIGRATION_COLUMNS to ALTER TABLE ADD COLUMN for older databases. It is additive; existing observations remain readable.

measurements(packet):

1. Starts with an empty result for every metric.
2. For each active/legacy metric, reads a finite value from the configured field.
3. Requires the sensor present and status valid.
4. Requires finger_on for optical metrics.
5. For wrist, requires verified contact, non-stuck raw, conversion within 0.02 °C, matching actual TMP117 status/value/raw/timestamp, and non-negative timestamp.
6. Applies the metric range.
7. Returns the value or None.

record(packet):

- uses time.time() (server time) unless a test supplies now;
- stores measurements() values; missing/invalid → SQL NULL;
- stores the wrist raw only when the wrist value is valid;
- stores finger_on and alert booleans;
- inserts a row for each accepted packet, even if values are null;
- prunes observations older than 30 days at most once per hour.

history(device_id, seconds=900):

- clamps seconds to 60..604800;

- bucket = max(1, seconds // 450);
- groups by bucket and device;
- averages active metrics, counts samples, maxes SOS/fall;
- fills every bucket from start to end, using null values for missing buckets;
- returns {device_id, bucket_seconds, points}.

The bucket start/end are computed from now and seconds, so a device disconnect creates null buckets rather than a shortened or interpolated line.

Server time versus device time: recorded_at is server time; device uptime_s/timestamps are also stored/available in the packet but history x-axis uses server time. The browser chart uses Date.now(); clock skew between server and browser could shift the visual window. This is a practical limitation, not a clinical feature.

C.7 Baseline and device views

- fetchBaseline() calls /api/baseline and /api/events.
- renderBaseline() shows enrollment status, person/alias, coverage days, ready metrics.
- renderReferenceTable() shows each active metric's qualified days, valid minutes, frozen model range or "Not established," current value and comparison.
- renderEvents() renders up to four recent events from up to 30 returned; it escapes event detail with esc().
- enrollment() POSTs start/end actions with the current device ID.
- renderReview() stores a clinician review note locally in the browser; it is not sent to the server or included in CSV/history.
- Device page shows device ID, connection/port, flash/PSRAM, uptime, history retention and sensor inventory.

The wrist baseline is context only. If wrist contact is unverified, the store has no wrist values, so a temperature baseline cannot qualify. This is deliberate.

C.8 Accessibility and presentation

- Skip link, landmarks, tab roles (role="tab", aria-selected), labels and keyboard ArrowLeft/ArrowRight handling.

- Chart has an `aria-label` with metric/time window/interval count.
- Responsive CSS and light/dark theme.
- Dynamic text is escaped in event/title paths (`esc()`), but this is not a formal XSS audit.
- No external CDN is required for the dashboard assets.

No full WCAG audit or real-browser accessibility test was performed.

D. Technical decisions and trade-offs

Decision	Reason	Trade-off
Pure helpers in `dashboard_core.js`	Testable without a DOM; consistent null/status/gap logic	DOM code still needs browser/E2E tests
Exact clinical temperature labels	Prevent actual sensor temperature from being read as wrist/core	OLED cannot show full labels; documented limitation
Separate actual TMP117 card	Actual sensor telemetry is useful without pretending contact is verified	More UI complexity; users must read labels
Clinical wrist null while unverified	Safety over visual completeness	Wrist card/baseline often unavailable
Null instead of zero for missing	Zero is a real value; missing is not zero	Chart/CSV/UI must handle null/blank
Gap-aware path	A line across a disconnect would imply continuous data	More segments/visual breaks; honest
Client-side CSV	No server file exposure; export current queried window	Depends on browser download; not audited for PHI
Request token for history	Prevents an older slow response overwriting a newer range	Slight extra state
SSE + periodic polling	Fast live view plus recovery/history refresh	Duplicate refreshes possible; `_received_at` reduces redundant render
SQLite WAL/RLock	Local concurrency and readable history	Not a multi-node/clinical database

Decision	Reason	Trade-off
Bucket averaging	Bounded history response and chart readability	Raw high-rate samples are not retained in history
Server-time recorded_at	One clock for DB/history queries	Browser/server clock skew can affect chart window
30-day retention	Bounds disk use	Old data is intentionally discarded
No actual TMP117 in history/CSV	Avoid treating sensor temperature as clinical wrist history	Actual trend is live-only, not historical
Additive migration	Preserve old observations	Schema still needs deliberate versioning for major changes
Local-only dashboard	Appropriate for prototype/demo	No auth/TLS; not network/PHI ready

D.1 Why the chart must not “fill the gap”

If a bucket has no valid observation, the chart could interpolate or carry the last value forward, producing a smooth line that implies monitoring occurred. The code instead starts a new path segment at the next valid point. The same honesty rule applies to CSV: missing values are blank, not zero.

E. Cross-module contributions and interfaces

E.1 Bridge → dashboard

- SSE payload shape: {status, telemetry, comparisons}.
- status includes connected/error/packets counters and optional storage_error.
- telemetry is a schema-2 packet with vitals, sensors, sensor_health, alerts, _received_at.
- /api/history returns bucketed points; /api/baseline and /api/events support the baseline view.
- The dashboard does not trust a value just because it arrived; DashboardCore.metricStatus() rechecks sensor/status/raw/range/evidence.

E.2 Storage → dashboard

- History points contain active metrics plus legacy `temp_object`; the dashboard only charts/exports active metrics.
- Missing history buckets are null; `samples=0`.
- `bucket_seconds` tells the UI the resolution.
- History is filtered by `device_id`, so switching devices does not mix patients.

E.3 Safety policy → dashboard

- `temperatureAssessment()` mirrors the safety policy: actual TMP117 separate, wrist unverified/null, core null, fever `not_available` with exact message.
- The exact message is surfaced at runtime from the assessment, not merely from static HTML.
- The baseline UI describes wrist as context only; it never presents a wrist model as core temperature.

E.4 Dashboard/storage contributions to state clearly

"My area turns accepted telemetry into a defensible display and a history that preserves absence: exact labels, separate actual versus clinical temperature, null instead of zero, and line breaks at gaps."

Do not claim authorship of the entire dashboard or database unless confirmed.

F. Evidence classification for this area

Claim	Classification	Verification
Exact temperature labels defined	[SOURCE]	dashboard.js metadata, dashboard_core.js SPECS, index.html
Actual/wrist separation rendered	[SOURCE] + [TEST]	renderEnvironment(), Node tests
Null/status/no-finger handling	[SOURCE] + [TEST]	metricValue(), metricStatus(), Node tests
Gap-aware chart	[SOURCE] + [TEST]	gapAwarePath(), Node test
CSV blank gaps/active columns	[SOURCE] + [TEST]	csv(), Node test

Claim	Classification	Verification
Four active metrics/no SHT/MLX UI	[SOURCE] + [TEST]	dashboard.js, activeSensorCount/sensor tests
SQLite schema/migration/retention	[SOURCE] + [TEST]	TelemetryStore, test_history.py
Verified-contact-only wrist storage	[SOURCE] + [TEST]	measurements(), history tests
Device isolation/gaps	[SOURCE] + [TEST]	history(), test_history.py
10 Node dashboard tests	[TEST] fresh 2026-10-29	node --test tests/dashboard_core.test.js
62 Python tests including history	[TEST] fresh 2026-10-29	python -m pytest -q
Full browser/E2E dashboard	[UNVERIFIED]	No browser automation/manual capture in this pass
WCAG/security audit	**Not performed**	No formal accessibility/security test
Real SQLite under disk-full/corruption/load	[UNVERIFIED]	Functional host tests only
Actual TMP117 history/CSV	**Not implemented by design**	History stores clinical wrist, not actual sensor field

F.1 Fresh checks

- 62 Python tests passed.
- 10 Node tests passed.
- Ruff/compile/JS syntax checks passed.
- Both PlatformIO builds succeeded (not required for dashboard logic, but current repository state).
- No browser screenshot/manual DOM/SSE test was performed in this documentation pass.

G. Debugging: dashboard/storage scenarios

G.1 “The wrist card is blank although TMP117 shows a value”

This is expected. The wrist card uses wrist_skin_temperature_status and wrist_contact_status. While contact is unverified, metricStatus() returns unverified and metricValue() returns null. The actual TMP117 card is separate. Do not "fix" this by copying actual into wrist.

G.2 "The chart line goes through a gap"

Check:

- /api/history points for null values;
- samples and bucket_seconds;
- DashboardCore.gapAwarePath() is being used;
- the current source has not been replaced by a naive L-only path.

The Node test chart path starts a new segment across a missing value guards this behavior.

G.3 "CSV has zero where data was missing"

That would be wrong. csv() uses finite() and outputs an empty string for null. Check that the CSV code path was not changed to point[column] || 0. Missing data should be blank; zero is valid only when actually measured.

G.4 "Old database fails after upgrade"

_migrate() adds the columns in MIGRATION_COLUMNS additively. Check PRAGMA table_info(observations), file permissions and whether the app is pointed at the intended data/telemetry.sqlite3. Do not delete the database to "fix" a migration without preserving evidence.

G.5 "History mixes devices"

history(device_id, ...) filters by device_id; the dashboard passes state.device. If devices mix, inspect the query/device selection, not the chart. test_persistence_device_isolation_and_wrist_raw covers store isolation.

G.6 "History chart is shifted"

History recorded_at uses server time; the chart window uses browser Date.now(). Check clock skew between the machine running the bridge and the browser. The timestamp is stored server-side; the chart is a visual query window.

G.7 “Dashboard is blank/offline”

- Is the bridge running and bound to localhost?
- Is `/api/stream` connected?
- Does `/api/telemetry` show fresh telemetry?
- After 5 s without a packet, the bridge/ dashboard intentionally show offline.
- A cleared packet is not a bug; it is the no-stale policy.

G.8 “Baseline never becomes ready”

Check the person/enrollment, 7 days, 30 sampled minutes per day, and active metric coverage. While wrist contact is unverified, wrist values are not stored; therefore overall readiness requiring all four metrics cannot complete. This is intentional. Do not inject actual TMP117 into the wrist column to make the baseline pass.

H. Testing and verification

H.1 Dashboard Node tests (10)

1. actual TMP117 sensor separate from unverified wrist contact;
2. verified contact exposes clinical wrist only with matching evidence;
3. non-valid status/offline/no-finger never substituted;
4. temperature assessment never invents core/fever;
5. verified assessment keeps core/fever unavailable;
6. stuck/missing/inconsistent evidence masked;
7. actual sensor can be valid outside clinical wrist range;
8. chart path starts a new segment across a missing value;
9. CSV exports clinical wrist data and leaves gaps blank;
10. sensor health and labels distinguish missing/valid.

H.2 Python history tests

- invalid/missing is not zero and zero is not missing;
- unverified contact never stored as wrist;
- only explicitly valid/verified statuses stored;
- missing raw cannot be a valid wrist temperature;
- persistence/device isolation/wrist raw;
- history keeps disconnect gaps.

H.3 What is not covered

- Actual browser DOM/event behavior, CSS layout, responsive rendering.
- SSE reconnect across a real network/browser.
- Chart tooltip/pointer accuracy in every browser.
- CSV download behavior/encoding in all browsers.
- Accessibility audit.
- SQLite disk-full/corruption/large-history performance.
- Security/XSS audit.
- Real patient data/privacy review.

H.4 Manual/browser checklist before claiming E2E

1. Start bridge with hardware/synthetic valid stream.
2. Open dashboard; confirm four cards and separate actual/wrist labels.
3. Confirm core/fever exact message and no SHT/MLX tabs.
4. Confirm wrist stays blank/unverified while actual TMP117 shows valid.
5. Disconnect and confirm null/offline, not stale values.
6. Reconnect and confirm recovery.
7. Confirm chart gap, tooltip, baseline band, CSV blank cells.
8. Test keyboard tab navigation, focus/contrast and responsive layout.

9. Confirm device isolation and 30-day retention behavior in a test DB.

10. Record versions/browser/OS and screenshots.

I. HOD preparation

I.1 60–90 second explanation

"My allocated area is the dashboard and local storage. The dashboard has four active metrics: heart rate, SpO2, wrist skin temperature and movement. The clinical wrist card is labelled exactly Wrist Skin Temperature in degrees Celsius, and the actual TMP117 reading is shown separately as TMP117 Sensor Temperature. While wrist contact is unverified, the wrist card stays blank with an unverified status; the actual sensor can still be valid. Core remains unavailable and fever shows the exact message that core temperature is unavailable and clinical confirmation is required. The history is SQLite; only valid observations are stored, unverified wrist values are never stored, and missing buckets remain null. The chart uses a gap-aware path so it never draws through a disconnect. CSV exports the active clinical metrics and leaves missing cells blank; it does not export the actual TMP117 sensor as wrist history. The ten Node tests and the Python history tests pass. I have not done a full browser end-to-end or accessibility/security audit, so I can defend the data/display rules, not claim production UI validation."

I.2 Technical questions and answers

1. **What are the four active dashboard metrics?** Heart rate, blood oxygen/SpO2, **Wrist Skin Temperature (°C)** and movement. The actual TMP117 sensor temperature is a separate card, not a fifth clinical metric.

2. **Where are the exact temperature labels defined?** In `assets/dashboard.js` metric metadata and `DashboardCore.SPECS`; `index.html` provides the accessible card labels. The OLED uses compact text and is not the exact-label surface.

3. **Why is the wrist card blank while TMP117 shows a number?** The TMP117 is actual sensor temperature; the clinical wrist value requires verified contact and matching evidence. Contact is always `unverified` in the current firmware, so `metricStatus()`/`metricValue()` return unverified/null for wrist.

4. **How does `metricStatus()` catch a bad actual TMP117 value?** It requires sensor presence, status, finite value, raw code, timestamp, non-stuck raw, value in `-55..150`, and signed conversion within `0.02 °C`. Otherwise it returns `sensor_error`.

5. **What evidence is required for a verified wrist value?** Verified contact, valid wrist status, raw/timestamp, non-stuck raw, conversion within 0.02 °C, actual TMP117 matching value/raw/timestamp, and wrist value in -10..60 °C.

6. **Why use null instead of zero for missing data?** Zero is a real measurement; missing is not zero. Using null prevents false baselines, charts and CSV values. The Node and Python tests explicitly distinguish zero from missing.

7. **How does the chart preserve gaps?** DashboardCore.gapAwarePath() resets started=false on a null value, so the next valid point begins with M rather than L. The line breaks instead of interpolating.

8. **What does CSV export contain?** Active metrics only: timestamp, heart rate, SpO2, clinical wrist skin temperature, motion, samples. Missing values are blank. It does not include actual TMP117 sensor history or legacy temp_object.

9. **What is historyRequest for?** It is a monotonically increasing token. A slower older /api/history response is ignored if a newer request has already started, preventing stale range data from overwriting the current chart.

10. **What does SQLite store, and what does it not store?** It stores valid active/legacy observation values, status context, raw wrist code, alerts and device ID. It does not store the actual TMP117 reading as the clinical wrist metric; missing/invalid values become SQL NULL.

11. **How does storage prevent unverified wrist values?** measurements() requires wrist_contact_status == "verified", non-stuck raw, valid status, matching actual TMP117 evidence/conversion and the -10..60 range; otherwise the wrist value is None.

12. **How is history bucketed and retained?** seconds is clamped to 60–604800; bucket is max(1, seconds // 450); missing buckets are null; observations older than 30 days are pruned at most hourly.

13. **What does the core/fever area show?** Core value "Unavailable," fever "Not available," and the exact runtime message **Core temperature unavailable — confirmation required**; no rule or core value is invented.

14. **Are SHT40/MLX90614 shown?** No. They are not installed and not part of the active inventory; the UI has no tabs/cards for them.

15. **What is missing from validation?** Full browser/DOM/SSE E2E, responsive/accessibility audit, security/XSS audit, disk-full/load/corruption testing and real patient/privacy review.

I.3 Five deeper follow-up questions

1. **"Why not store actual TMP117 in history for completeness?"** The clinical wrist history must not be contaminated by a package sensor reading. Actual TMP117 is live diagnostic telemetry; storing it in the wrist column would violate the safety model. A separate diagnostic table/column would need an explicit schema decision.

2. **"What happens if the browser clock and server clock differ?"** History buckets use server `time.time()`; the chart window uses browser `Date.now()`. Clock skew can shift the visible window. A production design would use server-provided window bounds or synchronize clocks.

3. **"How does the dashboard avoid drawing a false baseline band?"** It only draws the baseline band/model when `state.baseline.metrics[key].model` exists after seven qualified days. Otherwise it labels the range "not established." A missing model is not replaced with a guessed range.

4. **"Why is the full dashboard not covered by Node tests?"** `tests/dashboard_core.test.js` tests pure helpers, not DOM/SSE/download behavior. Browser E2E would require a browser automation harness; it has not been added. The source is reviewed, but that is weaker evidence than an executed browser test.

5. **"What would you change before using this with real patients?"** Add authentication/TLS, privacy controls, audit logs, browser E2E/accessibility/security testing, database resilience, time synchronization and clinical/regulatory review. The current local prototype is not a clinical record system.

I.4 Two cross-module questions

1. **"The bridge sends actual TMP117 valid and wrist unverified. What does the dashboard do?"** It shows the actual reading in **TMP117 Sensor Temperature (°C)**, blanks the clinical **Wrist Skin Temperature (°C)** card with unverified status, and keeps core/fever unavailable with the exact message. It does not copy the actual value into wrist.

2. **"How does the store preserve a disconnect in history?"** `history()` creates a point for every bucket from start to end; missing buckets get null values and `samples=0`. The chart's `gapAwarePath()` starts a new segment. CSV leaves corresponding cells blank because `finite(null)` is null.

I.5 Three improvements

1. **Add browser E2E/accessibility tests** (SSE updates, chart gaps, CSV download, keyboard/focus, responsive layouts) and record browser/OS evidence.

2. **Add a separate diagnostic history view/table for actual TMP117 sensor telemetry** if the team wants historical sensor trends, without ever mapping it to wrist/core.

3. **Harden storage and time handling:** server-provided window bounds/clock strategy, database backup/corruption/disk-full tests, and explicit retention/version policy for a larger deployment.

I.6 Ten key facts

1. Four active metrics: HR, SpO2, wrist skin temperature, motion.

2. Exact labels: **Wrist Skin Temperature (°C)** and **TMP117 Sensor Temperature (°C)**.

3. Actual TMP117 and clinical wrist are separate; wrist is blank/unverified now.

4. Core is unavailable; fever is `not_available` with the exact message.

5. Missing values are null/blank, never zero.

6. History is SQLite `data/telemetry.sqlite3`, 30-day retention, additive migration.

7. History buckets are null-filled; charts break at gaps.

8. CSV exports active clinical metrics only, blank gaps; no actual TMP history.

9. Wrist history requires verified contact and matching actual TMP117 evidence.

10. 10 Node tests + 62 Python tests pass; full browser/accessibility/security E2E is not done.

I.7 Five terms

Term	Explanation
Null-aware rendering	Displaying missing/invalid data as unavailable/blank, never substituting zero or a stale value.
Gap-aware path	SVG chart path that starts a new segment after a missing bucket.
Bucket	A fixed time bin used by `TelemetryStore.history()` to average observations and represent history compactly.

Term	Explanation
Additive migration	Adding missing columns without deleting/rewriting existing observations.
Clinical wrist temperature	The wrist-contact metric; currently unavailable and never derived from actual TMP117 sensor temperature.

I.8 Three source files to open first

1. `assets/dashboard_core.js` — `metricStatus()`, `metricValue()`, `temperatureAssessment()`, `gapAwarePath()`, `csv()`.

2. `assets/dashboard.js` — live consumption, exact metric labels, `renderEnvironment()`, `fetchHistory()`, `drawChart()`, CSV handler.

3. `telemetry_store.py` — `measurements()`, `record()`, `history()`, schema/migration/retention.

I.9 Three things never to claim without evidence

1. **"The dashboard is fully tested end-to-end/accessible/secure."** Only pure helper tests ran; no browser E2E/accessibility/security audit.

2. **"The wrist card shows a clinical wrist temperature."** It is blank/unverified; actual TMP117 is separate.

3. **"History/CSV contains complete continuous data."** Gaps are preserved as null/blank, and storage errors can mean missing records.

I.10 Responsibility checklist

- ☐ I can point to the exact label definitions and explain why OLED labels differ.
- ☐ I can explain the four active metrics and the separate actual/core/fever cards.
- ☐ I can trace a live SSE payload through `consume()`/`renderReadings()`.
- ☐ I can explain `metricStatus()`/`metricValue()` null/status behavior.
- ☐ I can explain verified wrist evidence requirements.
- ☐ I can show how `gapAwarePath()` breaks a line.
- ☐ I can describe history bucket/retention/device isolation.

- ☐ I can explain CSV columns, blank gaps and omission of actual TMP history.
- ☐ I can explain SQLite migration and server-time limitation.
- ☐ I can state the browser/E2E/accessibility/security testing gaps.
- ☐ I can answer the two cross-module questions without notes.
- ☐ I can state the three "never claim" boundaries without prompting.

J. Honest boundaries

J.1 What this member can defend

- The exact labels and separation of actual sensor versus clinical wrist.
- The null/status/no-finger display rules.
- The gap-aware chart and blank-gap CSV.
- SQLite schema, valid-observation storage, verified wrist requirements, retention and device isolation.
- The four-metric dashboard and absence of SHT40/MLX UI.
- The browser/DOM/E2E evidence limits.

J.2 What this member must not claim

- That a full browser dashboard test, accessibility audit or security audit was performed.
- That the wrist card currently shows a clinical wrist value.
- That actual TMP117 is stored/exported as wrist history.
- That history is continuous through disconnects.
- That storage is resilient to every disk/DB failure.
- That the local dashboard is network/PHI secure.
- That the chart is perfectly time-aligned across browser/server clock skew.
- That the dashboard/CSV are clinically validated.

J.3 Safe closing sentence

"The dashboard and store implement the project's honesty rules: exact labels, actual sensor versus clinical wrist separation, null instead of zero, gaps instead of interpolated lines, and verified-contact-only wrist storage. The helper and store tests pass, but full browser, accessibility, security and hardware end-to-end evidence still needs to be collected."